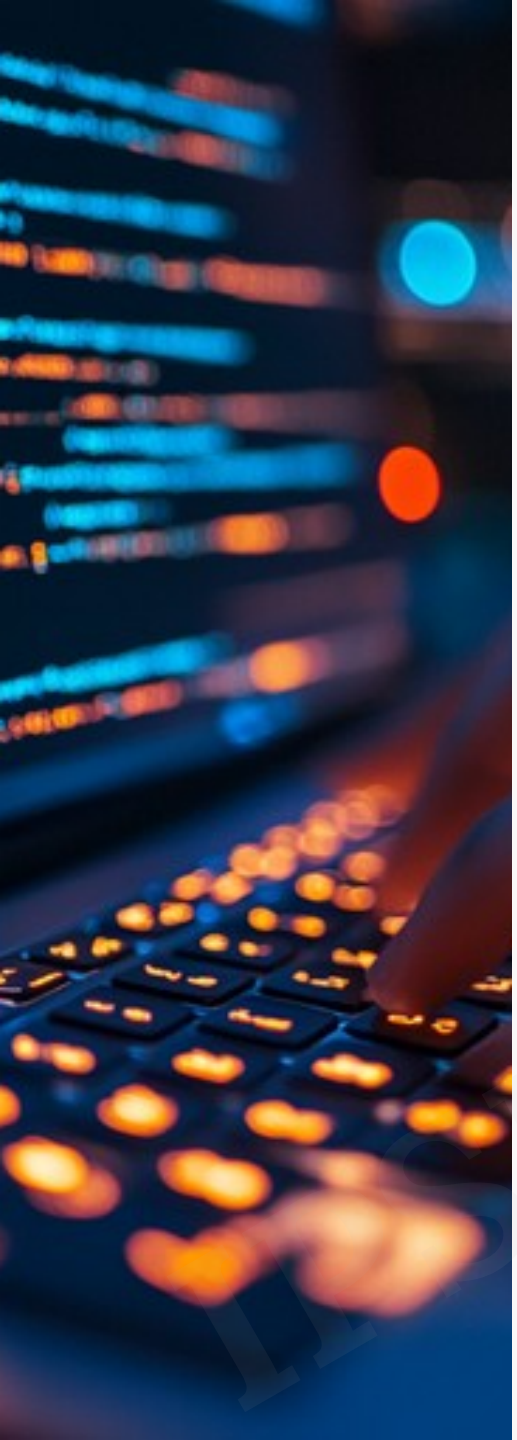


MSYS2 & Arm GNU Toolchain

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

Arm GNU Toolchain 與 Cross Toolchains

Arm GNU Toolchain

- **開放資源與官方維護**: Arm GNU 工具鏈是一套開放原始碼的完整開發工具集，涵蓋程式碼的編譯、連結到除錯的整個嵌入式開發流程。這套工具由 Arm 官方維護與發行，依不同發行版本提供對應的 Arm 架構、目標環境與指令集支援；是否支援特定核心，仍需查看該版本的發行說明。
- **核心元件**: 包含了 **GCC** 編譯器、**連結器 (Linker)**、**組合語言編譯器 (Assembler)**，以及功能強大的 **GDB** 除錯器等。
- **支援架構**: 完整支援 Arm 架構的 **32 位元 (AArch32)** 和 **64 位元 (AArch64)** 應用程式開發。

IPSI Lab, Yuntech-CSIE

Cross Toolchains

- 核心概念: 跨平臺工具鏈指的是在一個主機平台（**Host**）（例如：你的 Windows 或 Linux 開發機）上，編譯出可以跑在另一種不同架構或作業系統（**Target**）（例如：你的 Arm 晶片或嵌入式板）上的執行檔。
- 必要性: 由於許多嵌入式裝置（如微控制器或資源有限的單晶片）無法自己完成複雜的編譯工作，我們必須在效能更好的開發機上進行交叉編譯，這也是嵌入式系統開發的標準流程。

Arm GNU Toolchain 官方網站

- Arm GNU Toolchain（官方下載入口）

下載頁面

- Arm GNU Toolchain 安裝與版本選擇說明

Arm Toolchain 命名慣例 (Target Triple)

Arm 交叉工具鏈名稱常使用 Target Tuple（常稱 Target Triple） 描述目標架構、作業系統 / 執行環境與 ABI（Application Binary Interface，應用程式二進位介面）。名稱的欄位數及省略方式會因工具鏈而異；Host 平台需另外辨識。

常見概念： `目標架構 [–供應商] [–作業系統 / 環境] [–ABI]` （欄位可合併或省略，不是固定四段的格式）。

- 目標架構: `arm` ：指 32 位元 (AArch32)，常用於 Cortex-M 系列微控制器。 `aarch64` ：指 64 位元 (AArch64)，用於高階 Arm 處理器 (如 Cortex-A 系列)。
- 作業系統/環境: `none` ：表示 無作業系統 (Bare Metal) 或只搭配輕量級即時作業系統 (RTOS)，程式碼較精簡。 `linux` ：表示 Linux 目標環境；其 C 函式庫依工具鏈 / ABI 而定，常見 GNU/Linux 使用 glibc，但不能對所有 Linux 目標一概而論。

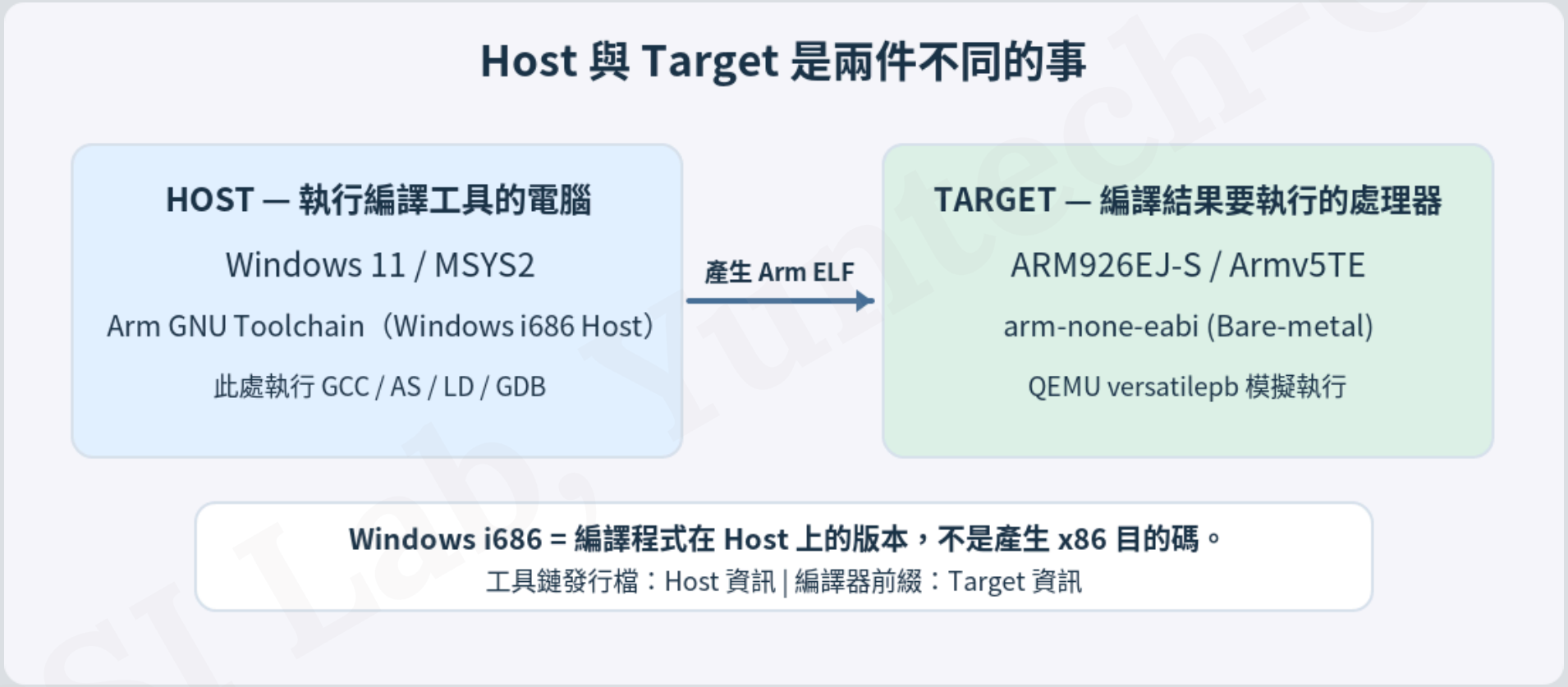
- **ABI (應用二進位介面)**: 定義了如何在程式碼中傳遞參數、如何調用函數等，以確保不同編譯器和軟體之間的相容性。例如 `none-eabi` 表示嵌入式無作業系統開發，`linux-gnueabihf` 支援硬體浮點（Hard-Float）運算的 Linux 開發。
- **Host（工具執行的平台）**：例如 `mingw-w64-i686` 表示可在 Windows 上執行的 32-bit 工具程式；這是發行壓縮檔名稱中的 Host 資訊，**不是 Target Triple 的一部分**。

常見命名範例

- `arm-none-eabi`：適用於 無作業系統 的 32 位元 Arm 微控制器 專案 (例如：開發樹莓派 Pico)。
- `aarch64-linux-gnu`：適用於 **64 位元 Arm Linux** 系統的應用程式開發 (例如：開發樹莓派 4/5 的 64 位元作業系統)。

工具鏈版本挑選與主機（Host）環境設定

Host 與 Target：不要把兩種架構搞混



重點： Windows/MSYS2 是 **Host**；`arm-none-eabi` 是 **Target**。Windows i686 是「編譯工具在 Windows 上執行的版本」，**不代表它產生 x86 目的碼**。

依據目標 CPU 架構與應用類型來挑選

目標 CPU/應用	Target Triple	典型應用情境 (台灣慣用稱呼)
32 位元 (Bare Metal)	`arm-none-eabi`	STM32、Raspberry Pi Pico (Cortex-M) 等**單晶片**專案，或是跑 **FreeRTOS** 等輕量化系統。
32 位元 (Linux 應用)	`arm-linux-gnueabihf`	運行 32 位元嵌入式 Linux 核心的裝置。`hf` 表示 hard-float 呼叫 ABI；**需與目標 CPU 的 FPU 能力及 Linux ABI 相容**，不可對所有 32-bit Arm Linux 裝置一律選用。
64 位元 (Bare Metal)	`aarch64-none-elf`	64 位元處理器的 **Bootloader** 或無 OS 的高階應用。
64 位元 (Linux 應	`aarch64-linux-gnu`	現行主流的 64 位元 Arm Linux 系統 (如 Arm 伺服器、樹莓派 64 位元系統)。

依據開發環境作業系統來挑選 (Host Toolchain)

Arm 官方下載檔案的格式為：

$$\text{arm-gnu-toolchain-}\langle\text{版本}\rangle\text{-}\langle\text{Host OS/Arch}\rangle\text{-}\langle\text{Target Triple}\rangle$$

- `Host OS/Arch`：指的是**開發環境**（您的 PC）的作業系統和 CPU 架構。
- `Target Triple`：指的是**目標嵌入式設備**的架構（`arm-none-eabi` 等）。

核心觀念： **你下載的 Toolchain (`Host OS/Arch`) 只是在你的電腦上執行的編譯程式；它編譯出來的程式 (`Target Triple`) 是給 Arm 晶片用的，兩者不會互相干擾。**

Windows 環境

- **64 位元作業系統**：
  - 選擇以 `mingw-w64-x86_64` 為基礎的Toolchain版本，如 `arm-none-eabi` 或 `aarch64-none-elf` ，適用於 64 位元 Windows。這是原生的 64 位元應用程式，在 64 位元 Windows 上執行效能最好。
  - 但目前ARM官方網站無此版本供下載，可從MSYS2（僅有 `arm-none-eabi` ）或其他來源或第三方Toolchain手動安裝。
    （**以課堂已驗證的 13.3.rel1 安裝包為基準**；如需安裝新版，應在官方實際下載清單確認 Host / Target，勿由命名格式自行推造檔名。）
  - 次要選擇: `mingw-w64-i686` 仍可運作於 64 位元 Windows (向下相容)，但它本質上是 32 位元的編譯程式。

- **32 位元作業系統（`mingw-w64-i686`）**
  - AArch32 bare-metal target (`arm-none-eabi`)，參考版本如 `arm-gnu-toolchain-13.3.rel1-mingw-w64-i686-arm-none-eabi`
  - AArch32 GNU/Linux target with hard float (`arm-none-linux-gnueabihf`)，參考版本如 `arm-gnu-toolchain-13.3.rel1-mingw-w64-i686-arm-none-linux-gnueabihf`
  - AArch64 bare-metal target (`aarch64-none-elf`)
  - AArch64 GNU/Linux target (`aarch64-none-linux-gnu`)

Linux 環境

- **x86_64（Intel/AMD CPU）**
  - AArch32 bare-metal target (`arm-none-eabi`)：應選 **Linux x86_64 Host** 的下載項目（例如檔名 Host 欄位為 `x86_64` ），不能誤用 Linux AArch64 Host 的 `...-aarch64-arm-none-eabi` 。
  - AArch32 GNU/Linux target with hard float (`arm-none-linux-gnueabihf`)
  - AArch64 bare-metal target (`aarch64-none-elf`)
  - AArch64 GNU/Linux target (`aarch64-none-linux-gnu`)
  - AArch64 GNU/Linux big-endian target（若該發行版本提供）：應以下載列表的 **`aarch64_be`** Target 項目為準；一般 `...aarch64-none-linux-gnu` **不是** big-endian 目標。

- **AArch64 （ARM CPU）**
  - AArch32 bare-metal target (`arm-none-eabi`)，參考版本如 `arm-gnu-toolchain-13.3.rel1-aarch64-arm-none-eabi`
  - AArch32 GNU/Linux target with hard float (`arm-none-linux-gnueabihf`)，參考版本如 `arm-gnu-toolchain-13.3.rel1-aarch64-arm-none-linux-gnueabihf`
  - AArch64 ELF bare-metal target (`aarch64-none-elf`)

IPSI Lab, Yuntech-CSIE

macOS 環境：

- **x86_64（Intel CPU）**
  - AArch32 bare-metal target (`arm-none-eabi`)，參考版本如 13.3.rel1 中的 `arm-gnu-toolchain-13.3.rel1-darwin-x86_64-arm-none-eabi`
  - AArch64 bare-metal target (`aarch64-none-elf`)，參考版本如 13.3.rel1 中的 `arm-gnu-toolchain-13.3.rel1-darwin-x86_64-aarch64-none-elf`
- **Apple silicon（M系列CPU）**
  - AArch32 bare-metal target (`arm-none-eabi`)，參考版本如 13.3.rel1 中的 `arm-gnu-toolchain-13.3.rel1-darwin-arm64-arm-none-eabi`
  - AArch64 bare-metal target (`aarch64-none-elf`)，參考版本如 13.3.rel1 中的 `arm-gnu-toolchain-13.3.rel1-darwin-arm64-aarch64-none-elf`

舉例

Raspberry Pi Pico（以 Windows 為開發環境）

- 使用 `arm-none-eabi` Target，適合 Cortex-M0+ 的 32 位元 Bare-metal 開發。
  Windows 64-bit 能執行某些 32-bit Host 工具，但 **Host 封裝名稱須以實際下載檔案為準**。
- 版本如 `arm-gnu-toolchain-13.3.rel1-mingw-w64-i686-arm-none-eabi`。
  對應格式：`arm-gnu-toolchain-ver.- Host OS/Arch - Target Triple`

IPSI Lab, Yuntech-CSIE

Raspberry Pi 4（以 Windows 為開發環境）

- **無作業系統開發**：若目標為 64-bit AArch64，可使用 `aarch64-none-elf` ；Windows Host 封裝需依該版本實際清單挑選，例如既有課程參考的 `...-mingw-w64-i686-aarch64-none-elf` 。
- **Linux 應用開發**：使用適用的 **AArch64 Linux Target**（例如 `aarch64-linux-gnu`）。原文 `...-x86_64-aarch64-none-linux-gnu` 的 `x86_64` 是 **Linux x86_64 Host**，不能列為 Windows 安裝包；實際可用組合依官方清單查核。

安装 Arm GNU Toolchain (Windows / MSYS2)

開發應用條件

- 本課程以 **Windows 11 / MSYS2 MINGW64** 與已驗證的 **Arm GNU Toolchain 13.3.rel1（Windows i686 Host、`arm-none-eabi` Target）** 為例；本機已成功執行的安裝方式保留。
- Target 為 `arm-none-eabi`；後續實際模擬機型為 QEMU `versatilepb`（ARM926EJ-S），**不是 Raspberry Pi Pico**。此處提及 Pico 僅作 Target 適用場景範例。

注意!!!

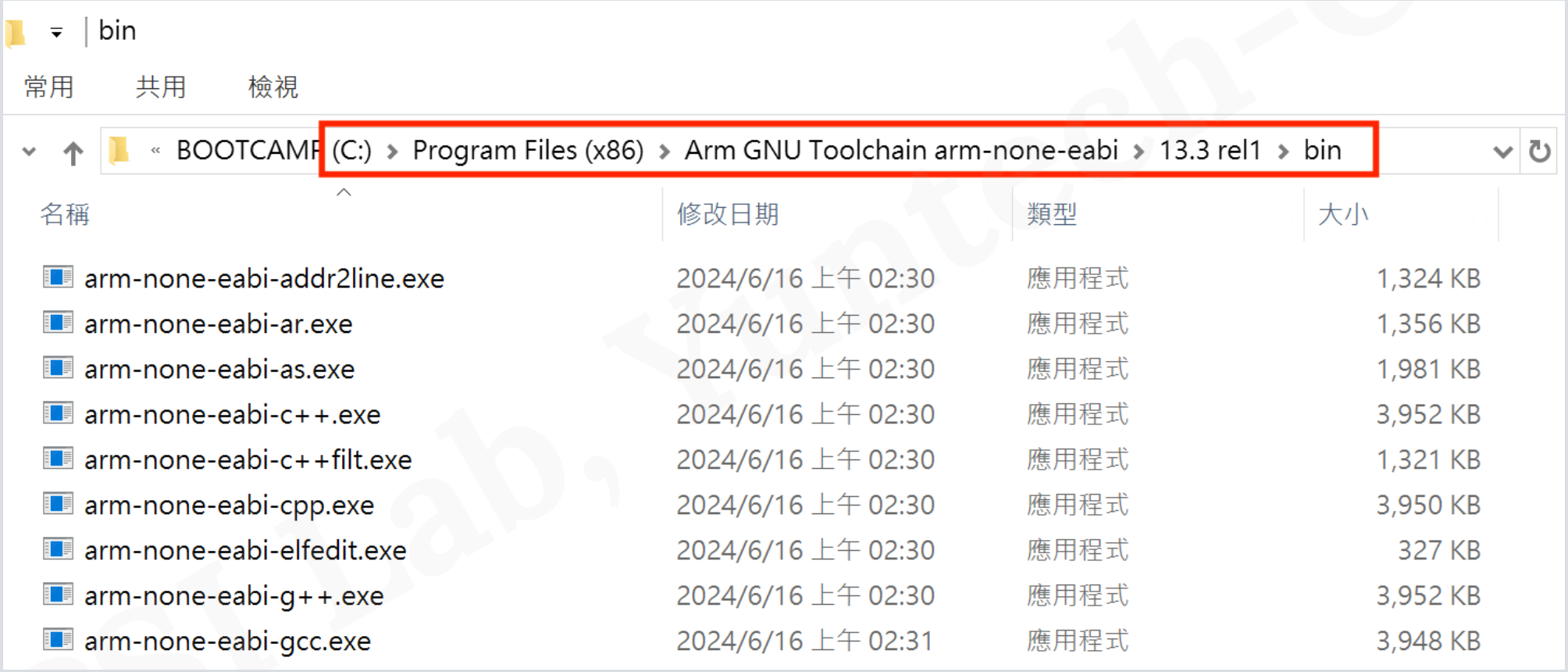
- 在此僅講述toolchain安裝，實際開發時還要依需求搭配相關套件與IDE。
- 如：開發Raspberry Pi Pico 時會再搭配其SDK與配置 IDE（如 VS Code）並安裝相應的插件和擴充套件，方便編譯與除錯。

1. 下載與安裝Toolchain

- 從 Arm Developer 網站下載 `arm-none-eabi` Toolchain版本，下載：`arm-gnu-toolchain-13.3.rel1-mingw-w64-i686-arm-none-eabi.exe` 。



- **安裝Toolchain後，記下安裝路徑 `bin` 資料匣**



2. 設定 MSYS2 環境變數（@MSYS2 `MINGW64` terminal）

- `.bashrc` 路徑:
  - `/home/user`
  - `C:\msys64\home\user`



IPSI Lab, Yuntech-CSIE

- 編輯 `.bashrc` 來設置 ARM Toolchain的路徑

```
nano .bashrc
```

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 ~
$ ls -a
.  ..  .bash_logout  .bash_profile  .bashrc  .profile
# ls 指令時，系統預設不會顯示以「.」開頭的隱藏文件，要加上`-a`

lijw@DESKTOP-5B5R93E MINGW64 ~
$ nano .bashrc
# 以nano打開進行編輯
```

IPSI Lab, Yuntech-CSIE

- 在 `.bashrc` 中加入 `bin` 資料匣路徑後儲存。

```
export PATH="$PATH:/C/Program Files (x86)/Arm GNU Toolchain arm-none-eabi/13.3 rel1/bin"
```

❖ 需根據Toolchain實際安裝路徑修改

- 讓設定生效

```
source ~/.bashrc
```

IPSI Lab, Yuntech-CSIE

3. 驗證Toolchain是否安裝成功（@MSYS2 `MINGW64` terminal）

```
arm-none-eabi-gcc --version
```

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 ~
$ arm-none-eabi-gcc --version
arm-none-eabi-gcc.exe (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 13.3.1 20240614
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

IPSI Lab, Yuntech-CSIE

安装 QEMU（Windows / MSYS2）

QEMU（Quick Emulator）

- 開放原始碼的模擬器與虛擬化工具
- 支援多種硬體架構模擬，包括 **x86**、**ARM** 等
- 常用於**跨平台開發**和嵌入式系統測試

IPSI Lab, Yuntech-CSIE

QEMU 的核心功能

- 模擬器功能
  - 在不同架構間執行應用程式
  - 可在 **x86 主機**上模擬 **ARM 系統**
- 虛擬化功能
  - 在 **Host / Guest 架構與虛擬化能力相容**時可結合 **KVM（Kernel-based Virtual Machine）**；本課程在 Windows 上以 QEMU 模擬 Arm 系統，**不屬於 KVM 加速情境**。
  - 適用於高效能虛擬化需求

IPSI Lab, Yuntech-CSIE

QEMU 的應用

- **嵌入式系統開發**
  - 減少實體硬體需求，適合初期測試和調試
  - 支援基礎的 GPIO、UART 等外設模擬
- **跨平台測試**
  - 提供多種系統架構模擬環境，便於跨平台軟體開發

IPSI Lab, Yuntech-CSIE

在 MSYS2 安裝 QEMU（MINGW64）

- 更新 MSYS2 packages，確保是最新的，以避免安裝過程中出現相容性問題。

```
pacman -Syu
```

- 安裝既有課堂 `MINGW64` 環境所使用的 QEMU 套件（以下命令保留原實驗流程）

```
pacman -S mingw-w64-x86_64-qemu
```

Info.: Package: mingw-w64-x86 64-qemu - MSYS2 Packages

- **確認安裝**

```
qemu-system-arm --version
```

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 ~
$ which qemu-system-arm
/mingw64/bin/qemu-system-arm
# (option) 查看路徑

lijw@DESKTOP-5B5R93E MINGW64 ~
$ qemu-system-arm --version
QEMU emulator version 9.1.1
Copyright (c) 2003-2024 Fabrice Bellard and the QEMU Project developers
```

2026 年環境更新提醒： MSYS2 已公告逐步停用 `MINGW64` 套件建置，建議**新安裝環境**優先評估 `UCRT64`（對應套件：`mingw-w64-ucrt-x86_64-qemu`）。**既有 MINGW64 實驗紀錄與指令保留，不強制遷移**；新舊環境的 QEMU 路徑可能不同。

實作實務提示

MSYS2運行/開發環境的版本

- `MINGW64` **Terminal**
  - 是基於 MSVCRT（Microsoft Visual C Runtime），因為它基於 MSVCRT 編譯，並使用 Microsoft Visual C Runtime，所以需要基於 MSVCRT編譯成的應用程式。
  - 雖然是基於 64 位元的 MSVCRT 執行環境，但它仍然支援執行 32 位元的應用程式（向下相容）。
  - EX., prefix: `mingw-w64-x86_64-` 、 `mingw-w64-i686-`

IPSI Lab, Yuntech-CSIE

- **`UCRT64` Terminal**
  - 是基於 UCRT（Universal C Runtime），因為它基於 UCRT 編譯，並使用 Universal C Runtime，所以需要基於 UCRT編譯成的應用程式。
  - 雖然是基於 64 位元的 UCRT 執行環境，但它仍然支援執行 32 位元的應用程式（向下相容）。
  - EX., prefix: `mingw-w64-ucrt-x86_64-` 、 `mingw-w64-ucrt--i686-`

IPSI Lab, Yuntech-CSIE

QEMU版本與運行時環境有關

1. **UCRT 和 MSVCRT 不同的運行時環境**
   - `MINGW64` **Terminal**需要 MSVCRT 版本的應用程式（例如 `mingw-w64-x86_64-qemu`，這樣才能與 MSVCRT 運行時相容。
   - `UCRT64` **Terminal**需要 UCRT 版本的應用程式（例如 `mingw-w64-ucrt-x86_64-qemu`）。
2. **避免運行時不相容問題**
   - 在不同 Terminal 混用套件時，可能遇到 DLL 搜尋路徑或執行環境衝突；應確認實際可執行檔路徑、套件與相依函式庫。**不能單由 Terminal 名稱斷定另一環境的 EXE 必定無法執行**。

Toolchain的目標架構與開發環境無關

EX. `arm-gnu-toolchain-13.3.rel1-mingw-w64-i686-arm-none-eabi`

- Toolchain的版本，如：AArch32 bare-metal target (`arm-none-eabi`)，是針對 ARM 架構的目標生成工具，與 Windows 主機環境的位元無關。
- 無論在 `UCRT64` 還是 `MINGW64` Terminal中，Toolchain生成的二進位檔案都是針對 ARM 而非 Windows 系統，因此不受執行環境的限制。

IPSI Lab, Yuntech-CSIE

無作業系統（bare metal）開發 `arm-none-eabi` 無法提供標準系統呼叫

- 無作業系統，無法提供標準系統呼叫
- `printf()` 會透過 C 函式庫的輸出介面呼叫底層 `_write()`；`malloc()` 則可能透過 `_sbrk()` 取得 Heap 空間。**`printf` / `malloc` 本身不是系統呼叫**。
- 本課程已有可執行的 `syscalls.c`：其中 `_write()` 將字元寫入 `versatilepb` UART0（位址 `0x101F1000`），所以 `printf()` 能在 QEMU 終端顯示。這是**特定模擬板的簡化支援程式**，不是 Bare-metal 內建的標準輸出；目前不修改。
  - syscalls.c 函數列表：`_close`、`_fstat`、`_isatty`、`_lseek`、`_read`、`_getpid`、`_kill`、`_write`、`_sbrk` 等。

編譯/組譯範例操作

操作1：編譯 Arm C 程式

1. 加入 `syscalls.c` (與程式同資料匣)

2. `main.c`

```c
#include <stdio.h>

int main() {
int i;
for (i = 0; i < 10; i++) {
    printf("Loop iteration: %d\n", i);
}
printf("Program finished.\n");
return 0;
}
```

IPSI Lab, Yuntech-CSIE

3. 編譯程式

```
arm-none-eabi-gcc -o main.elf main.c syscalls.c -specs=nosys.specs -lc -lgcc
```

- `arm-none-eabi-gcc`：用於 Arm Bare-metal 目標的交叉編譯器；本課程實驗的 QEMU `versatilepb` 預設為 **ARM926EJ-S（Armv5TE）**，而不是 Armv7 / Armv8 核心。
- `-o main.elf`：指定輸出檔案名為 `main.elf`（`ELF` 格式的可執行檔）
- `-specs=nosys.specs` 選項
  - 使用適合 Bare-metal / Newlib 的連結規格，通常會納入 `libnosys` 提供未實作系統呼叫的替代 stub。
  - **不代表禁用所有 C 函式庫，也不會強制覆蓋函式定義**；實際採用哪個 `_write` / `_sbrk`，依物件檔與函式庫的符號解析、連結順序決定。
  - 本課程同時編入 `syscalls.c`，使用自己的 UART `_write()`；目前成功輸出的實驗紀錄保留。
  - **可驗證：** `arm-none-eabi-nm main.elf | grep "_write\|_sbrk"`（檢查符號，不代表單看此命令就能判斷全部執行語意）。

4. 使用 QEMU 模擬程式

```
timeout 1s qemu-system-arm -M versatilepb -m 128M -kernel main.elf -nographic
```

- 命令執行流程
  - 設定 1 秒執行限制防止模擬器卡死
  - 啟動 QEMU 模擬器，載入 main.elf 並執行
  - 輸出結果直接顯示在命令行，便於偵錯與觀察

IPSI Lab, Yuntech-CSIE

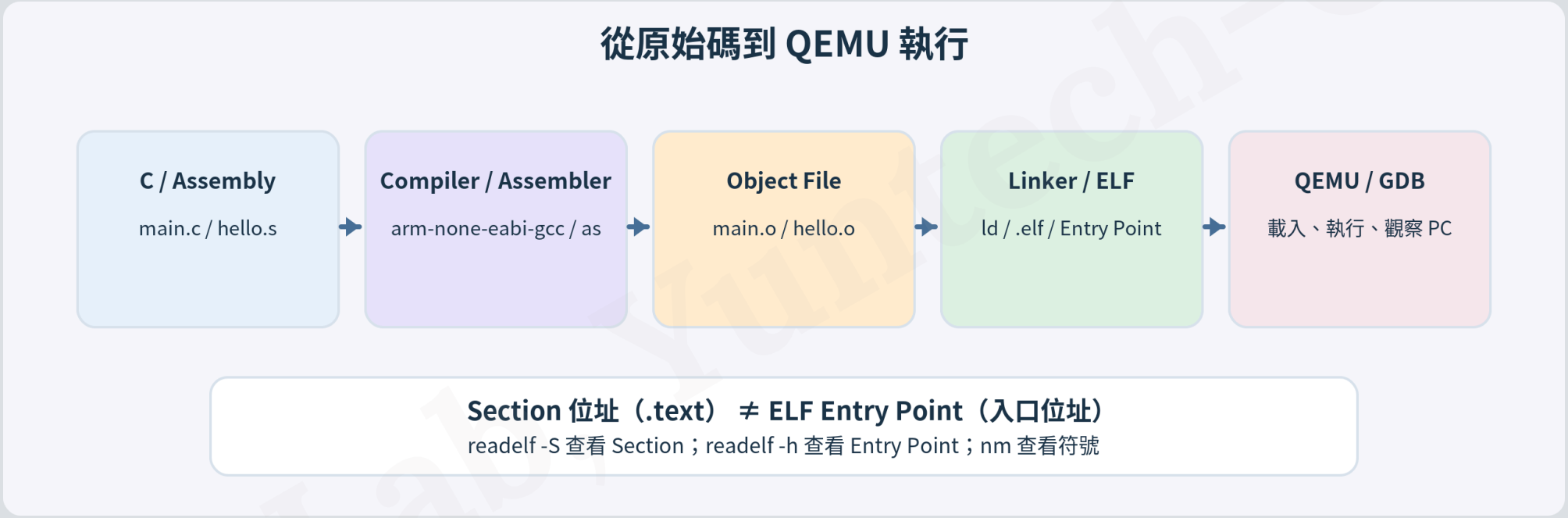
- timeout 1s：設定執行超時時間為 1 秒，防止模擬卡住
- qemu-system-arm：QEMU 的 ARM 系統模擬器，用於執行 ARM 架構的程式
- -M versatilepb 選項
  - 指定硬體機型為 versatilepb（Versatile Platform Board）
  - 模擬一款常見的 ARM 虛擬開發板，適合初學者使用
  - 包含 UART 等基本外設，便於無作業系統開發與測試
- -m 128M 選項
  - -m 128M：設定模擬系統的記憶體大小為 128 MB
- -nographic：禁用圖形界面，僅使用命令行介面進行模擬
- `-kernel main.elf`：QEMU 依**此板機模型的映像載入 / 開機流程**處理執行檔；ELF Entry Point、載入區段與初始化仍應以工具及 GDB 驗證，不能把所有 QEMU 機型都視為同一個直接設定 PC 的流程。

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/c
$ timeout 1s qemu-system-arm -M versatilepb -m 128M -kernel main.elf -nographic
Loop iteration: 0
Loop iteration: 1
Loop iteration: 2
Loop iteration: 3
Loop iteration: 4
Loop iteration: 5
Loop iteration: 6
Loop iteration: 7
Loop iteration: 8
Loop iteration: 9
Program finished.
```

IPSI Lab, Yuntech-CSIE

編譯、連結與載入不是同一件事



重點： **Assembler / Compiler 產生 `.o`；Linker 產生 ELF 並記錄 Entry Point；QEMU 依板機設定載入，GDB 可觀察真正開始執行的位置。**

操作2：組譯 Arm 程式（組合語言）

1. `hello.s`

此範例為 **Armv5TE / A32** 的裸機 UART 輸出片段，使用 `versatilepb` 上的 PL011 UART0（`0x101F1000`）。
程式碼中的 `//` 為原始實驗註解；若希望跨 GNU Assembler 版本更一致，可改採 A32 常見的 `@` 行內註解。

IPSI Lab, Yuntech-CSIE

```
.section .data
msg:
.asciz "Hello, World!\n"

.section .text
.global _start

// Base address of the UART0
.equ UART0, 0x101f1000

_start:
ldr r1, =msg        // 將訊息地址載入 r1
bl print_string     // 呼叫輸出函數

// 結束程式
b .

print_string:
ldrb r2, [r1], #1     // 每次從 r1 (msg) 中讀取一個字元到 r2
cmp r2, #0            // 檢查是否到字串結尾
beq end_string        // 如果是 0 (字串結尾)，跳轉到 end_string

// 等待 UART 傳輸可用
wait_uart:
ldr r3, =UART0        // 設定 UART 地址
ldr r4, [r3, #0x18]   // 讀取 UARTFR (Flag Register)
tst r4, #0x20         // 檢查 TXFF (Transmit FIFO Full)，若為 1 則等待
bne wait_uart

// 發送字元
str r2, [r3, #0x00]   // 寫入字元到 UARTDR (Data Register)
b print_string        // 迴圈繼續發送字串中下一個字元

end_string:
bx lr                 // 返回
```

IPSI Lab, Yuntech-CSIE

2. 組譯

```
arm-none-eabi-as -o hello.o hello.s
arm-none-eabi-ld -Ttext=0x10000 -o hello.elf hello.o
```

- `arm-none-eabi-as` 是 ARM 架構的組譯器，用於將組合語言程式（`.s` 檔案）轉換為目標檔案（`.o` 檔案）
- `arm-none-eabi-ld` 是 ARM 的連結器，用於將多個目標檔案（`.o` 檔案）連結成一個可執行檔案（如 `.elf` 檔案）
- `-Ttext=0x10000`：設定 `.text` **輸出區段**位址為 `0x10000`；**不等於單獨指定 ELF Entry Point**。本範例由預設連結入口慣例解析 `_start`；若需明確指定，可額外使用 `-e _start`，但不更動已成功的課堂指令。
- **檢驗差異：** `arm-none-eabi-readelf -h hello.elf` 查看 Entry Point；`arm-none-eabi-readelf -S hello.elf` 查看 Section 位址；`arm-none-eabi-nm hello.elf` 查看 `_start` 符號。

3. 使用 QEMU 模擬程式

```
timeout 1s qemu-system-arm -M versatilepb -m 128M -nographic -kernel hello.elf
```

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/as
$ arm-none-eabi-as -o hello.o hello.s

lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/as
$ arm-none-eabi-ld -Ttext=0x10000 -o hello.elf hello.o

lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/as
$ timeout 1s qemu-system-arm -M versatilepb -m 128M -nographic -kernel hello.elf
Hello, World!
```

IPSI Lab, Yuntech-CSIE

操作3：編譯 C 與組合語言混用（C 呼叫組合語言）

1. 檔案結構：

- `main.c`：C 程式碼
- `functions.s`：組合語言程式碼

2. `syscalls.c`

IPSI Lab, Yuntech-CSIE

3. 編譯與組譯

```
arm-none-eabi-gcc -o main.elf main.c syscalls.c functions.s -specs=nosys.specs -lc -lgcc
```

4. 使用 QEMU 模擬程式

```
timeout 1s qemu-system-arm -M versatilepb -m 128M -nographic -kernel main.elf
```

IPSI Lab, Yuntech-CSIE

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/ca2
$ ls
functions.s  main.c  syscalls.c

lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/ca2
$ arm-none-eabi-gcc -o main.elf main.c syscalls.c functions.s -specs=nosys.specs -lc -lgcc

lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/ca2
$ timeout 1s qemu-system-arm -M versatilepb -m 128M -nographic -kernel main.elf
3 + 7 = 10
5! = 120
Before swap: x = 10, y = 20
After swap: x = 20, y = 10
```

IPSI Lab, Yuntech-CSIE

main.c

```c
#include <stdio.h>

// 聲明組合語言中的函數
extern int add_numbers(int a, int b);
extern int factorial(int n);
extern void swap_numbers(int *a, int *b);

int main() {
    // 測試 add_numbers 函數
    int sum = add_numbers(3, 7);
    printf("3 + 7 = %d\n", sum);

    // 測試 factorial 函數
    int fact = factorial(5);
    printf("5! = %d\n", fact);

    // 測試 swap_numbers 函數
    int x = 10, y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap_numbers(&x, &y);
    printf("After swap: x = %d, y = %d\n", x, y);

    return 0;
}
```

IPSI Lab, Yuntech-CSIE

functions.s

```
// functions.s
.global add_numbers       @ 讓這些符號可以被外部存取
.global factorial
.global swap_numbers

.section .text
@ 函式：add_numbers
@ 參數：r0 = a, r1 = b
@ 返回：r0 = a + b
add_numbers:
    add r0, r0, r1        @ 將r0和r1相加，結果存回r0
    bx lr                 @ 返回呼叫者

@ 函式：factorial
@ 參數：r0 = n
@ 返回：r0 = n!
factorial:
    push {r4, lr}         @ 保存本層 R4 與 LR（共 8 bytes）
    mov r4, r0            @ 保存n
    cmp r0, #1            @ 比較n和1
    ble fact_end          @ 此既有範例只處理 n>=1；n<=1 就返回 n
    sub r0, r0, #1        @ n = n - 1
    bl factorial          @ 遞迴呼叫
    mul r0, r4, r0        @ 將結果與n相乘
fact_end:
    pop {r4, lr}          @ 恢復本層 R4 與 LR
    bx lr                 @ 返回

@ 函式：swap_numbers
@ 參數：r0 = &a的位址, r1 = &b的位址
swap_numbers:
    ldr r2, [r0]          @ 載入 *a 到 r2
    ldr r3, [r1]          @ 載入 *b 到 r3
    str r2, [r1]          @ 存 r2 到 *b
    str r3, [r0]          @ 存 r3 到 *a
    bx lr                 @ 返回
```

IPSI Lab, Yuntech-CSIE

既有實驗相容性： 此處 `factorial` 以 **正整數** `1 <= n <= 12` 為教學輸入，`factorial(5)=120` 已有成功紀錄；**目前對** `n=0` **會回傳 0（不是數學上的** `0!=1`**）**。為使後續 GDB 位址與反組譯紀錄能對照，先**不改動可執行的演算法路徑**；日後若統一支援 `0!`，須同步重建 GDB 範例。

IPSI Lab, Yuntech-CSIE